

SETUP GUIDE

Your AI job-search assistant

How to set up career-ops with daily job discovery, a dashboard of everything you've applied to, Gmail status sync, and assisted application filling, step by step from a fresh Mac.

Twice-daily job scan

Fit-ranked digest

One-click "Mark Applied"

Gmail rejection & interview sync

Form filling that never submits

Contents

1. What you're setting up
2. Install the basics
3. Download the project
4. First run: let Claude onboard you
5. Tell it what you're looking for
6. Your answer bank
7. Your final CV
8. Optional extra job sources
9. Schedule the daily scan
10. The dashboard
11. Gmail sync
12. Applying to a job
13. Your house rules
14. A normal day
15. Staying up to date
16. Privacy
17. Troubleshooting

Project: github.com/basharatfaiz/career-ops, built on the open-source **career-ops** project (github.com/career-ops-hq/career-ops). Technical reference inside the project: `docs/EXTRAS.md`.

STEP 1

What you're setting up

career-ops is a folder on your Mac that an AI assistant (Claude Code) works inside. You talk to it in plain English, and it runs small scripts that find jobs, score them against your CV, and keep a tracker. Nothing lives on anyone else's server.

Daily discovery

Twice a day it scans company career pages, job boards and (optionally) LinkedIn, filters out roles that don't fit, and sends you a Mac notification.

Digest & dashboard

A page in your browser listing new roles ranked by fit, everything you've applied to, and full history, with Mark Applied and Archive buttons.

Gmail sync

Reads your job-email labels and proposes tracker updates (rejected, interview, offer). You approve each one.

Assisted applying

Opens an application in a visible browser and fills it from your own saved answers and your one final CV, then stops so you can review and submit yourself.

It never applies on your behalf.

Every tool stops before the Submit button, and nothing is marked "Applied" unless you say so. Answers it can't back up from your own files are handed back to you as questions, never guessed.

Time needed

- **About 45 minutes** for the first setup (steps 2–9).
- **15 more minutes** for Gmail sync (step 11), if you want it.
- After that, about 10 minutes a day.

What you need

- A Mac. The daily scan and notifications use macOS. Linux works with small changes; see Troubleshooting.
- A Claude plan that includes **Claude Code** (the Claude desktop app's Code tab, or the `claude` terminal app).
- Your CV as a PDF, plus its text (you'll paste it in once).
- Optional: a free GitHub account, only if you later want to share your own improvements.

STEP 2

Install the basics

Three free tools: Homebrew (installs the others), Node.js (runs the scripts) and Git (downloads the project). Open the **Terminal** app (press ⌘ Space, type "Terminal") and paste each command, pressing Return after each.

a Install Homebrew

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password (typing shows nothing; that's normal). When it finishes, it prints two "Next steps" commands. Run those too, so `brew` works.

b Install Node.js and Git

```
brew install node git
```

Check they work. Both should print a version number (Node must be 18.17 or newer):

```
node -v
git --version
```

c Get Claude Code

Easiest: install the **Claude desktop app** and use its **Code** tab, which lets you open a folder and chat with Claude about it. Alternatively, install the terminal version:

```
npm install -g @anthropic-ai/claude-code
```

Sign in with your Claude account the first time you open it.

Prefer not to touch the Terminal?

Once Claude Code is installed (step c), you can open it on any folder and ask: *"Install Homebrew, Node and Git for me."* It will run the commands and ask your permission for each one.

STEP 3

Download the project

This copies the project into a folder called `career-ops` in your home folder and installs what it needs.

```
cd ~  
git clone https://github.com/basharatfaiz/career-ops.git  
cd career-ops  
npm install  
npx playwright install chromium  
cp docs/local-paths.fork.txt config/local-paths.txt
```

The `playwright` command downloads a private copy of the Chromium browser that the tools use to read job pages and fill forms. It's separate from your normal Chrome, so your own browsing, logins and passwords are never touched.

The last line tells the project's built-in updater that the extra files in this version belong to you, so a future update never overwrites them.

Where is it?

In Finder: your home folder (Go → Home) → `career-ops`. You'll open this folder in Claude Code next.

STEP 4

First run: let Claude onboard you

Open the `career-ops` folder in Claude Code (desktop app: Code tab → choose folder; terminal: `cd ~/career-ops` then `claude`). Then just say:

"Hi, set me up."

It checks what's missing and walks you through it, one question at a time:

IT ASKS FOR	WHAT TO GIVE IT	SAVED TO
Your CV	Paste the text of your CV. It turns it into clean markdown.	<code>cv.md</code>
Profile	Name, email, location, timezone, target roles, salary range, and a cost tier (pick standard if unsure).	<code>config/profile.yml</code>
Job sources	Say yes to the pre-configured company list; it tailors the keywords to your target roles.	<code>portals.yml</code>
About you	Your strengths, deal-breakers, best achievement. The more honest detail, the better it filters.	<code>modes/_profile.md</code>

Important: personalise your profile.

If Claude says *"modes/_profile.md is still the shipped template"*, say yes when it offers to personalise it from your CV. Otherwise every job is scored against the template author's targeting instead of yours.

Check it worked: ask *"Run the setup doctor."* It should report that nothing is missing.

STEP 5

Tell it what you're looking for

Two filters decide which jobs ever reach you: **job titles** and **location**.

Job titles

Titles are filtered by keyword in `portals.yml` → `title_filter`: **positive** titles to keep, **negative** words to drop (e.g. "Intern", "Director"). Just tell Claude in plain words:

"I'm looking for senior individual-contributor product design roles. Exclude manager, lead, graphic and marketing design."

Searching for product design roles too?

The project includes a ready-made filter: `tests/fixtures/title-filter.product-design.yml`. Ask Claude: *"Use the product-design title filter from tests/fixtures in my portals.yml."*

Location

The location rules assume you live in **India**: India-based and India-eligible remote roles are kept, and roles abroad are kept only as "relocation" options. If you live elsewhere, ask:

"I'm based in [country]. Change the location rules and job-source location lists to match."

The relevant files are listed under "Defaults to change" in `docs/EXTRAS.md`.

Seniority

A seniority gate rejects Staff, Principal, Lead, Manager and Director titles, and visual, graphic or marketing design roles. If you *want* lead roles, tell Claude to relax the seniority gate.

STEP 6

Your answer bank

Application forms ask the same questions over and over: notice period, expected salary, work authorisation, "how did you hear about us". The answer bank is the one file where *you* write those answers once. The form filler uses only this file and your CV.

```
cp templates/application-answers.example.yml data/application-answers.yml
```

Then open it with Claude: *"Help me fill in my answer bank, one section at a time."*

SECTION	WHAT GOES IN IT
identity	Name, phone, email, LinkedIn, portfolio link.
compensation	Current and expected salary, exactly as you want them stated.
availability	Notice period. Joining date is always asked, never guessed.
work_authorization	Where you may work and whether you need sponsorship.
location_preference	Your current city and country, and whether you're comfortable with hybrid/onsite (onsite_comfort).
demographics	Optional. Only answered if you fill it in. List anything you never want answered under never_answer .
verified_experience	Employers, metrics, tools, education, copied from your CV.
free_text	Your own short answers for "Why this company?", "Tell us about a project you're proud of" and so on: highlights , angles and stories , written in your own words.

Blank is fine, and safe.

Any field you leave empty makes the form filler stop and ask you, rather than guess. Start with identity, compensation and availability, and fill in the rest over time.

Stories use placeholders.

In free_text.stories you can write {company} , {role} or {termA} (a key product term from the job post), and they're filled in per application. The generator never invents experience or numbers: every figure must already be in your CV.

STEP 7

Your final CV

You keep **one** polished PDF CV that you designed yourself. The tools upload exactly that file, every time, and refuse any other.

1. Put your CV in the project, e.g. `documents/cv/final/Your Name – CV.pdf`
2. Get its fingerprint (a hash, so a different or edited file is never uploaded by mistake):

```
shasum -a 256 "documents/cv/final/Your Name – CV.pdf"
```

3. In your answer bank, set `cv.final_path` to the file path and `cv.final_sha256` to the long code it printed.
4. Check it:

```
node final-cv.mjs
```

It should say the CV is verified.

Updated your CV?

Replace the PDF, run `shasum` again and update the hash. Until you do, the tools refuse to upload the changed file, on purpose. Also ask Claude to update `cv.md` so job scoring uses the new content.

Want Claude to never generate or tailor CVs for you? Add that to your house rules (step 13).

STEP 8 · OPTIONAL

Optional extra job sources

Out of the box, discovery uses free sources: company career pages (Greenhouse, Lever, Ashby, Workday) and a public-search sweep. Two paid services add more. Both are off until you add a key, and both have small spend caps built in.

SERVICE	ADDS	KEY IN <code>.ENV</code>
Apify	LinkedIn job listings (used only to <i>find</i> roles, then matched to the employer's own page).	<code>APIFY_TOKEN=...</code>
Firecrawl	Web search and reading of JavaScript-heavy career pages.	<code>FIRECRAWL_API_KEY=...</code>

```
cp .env.example .env
```

Then paste your key after the `=` on the right line. The `.env` file is private and never uploaded anywhere.

Keep keys secret.

Never paste an API key into a chat, an email or a shared document. If one leaks, delete it in that service's dashboard and create a new one.

STEP 9

Schedule the daily scan

First, run one scan by hand to make sure everything works (it can take several minutes):

```
bash daily-scan.sh
```

You should get a Mac notification like "career-ops: 7 new role(s)", and `output/daily-digest.html` will exist.

Make it run twice a day automatically

The easy way is to ask Claude:

"Schedule daily-scan.sh to run at 11:30 and 20:00 every day using launchd, using the plist in docs/EXTRAS.md."

Or do it yourself: save the plist from `docs/EXTRAS.md` (step 5 there) as `~/Library/LaunchAgents/com.careerops.daily-scan.plist`, replace `/ABSOLUTE/PATH/TO/career-ops` with your real path (find it by running `pwd` inside the folder), then:

```
launchctl load ~/Library/LaunchAgents/com.careerops.daily-scan.plist
```

Your Mac must be awake.

A scheduled scan runs only if your Mac is on. If it was asleep, the scan runs at the next scheduled time. To stop the schedule, run the same command with `unload` instead of `load`.

STEP 10

The dashboard

The digest is a page you open in your browser. To make its buttons work, start the small local server:

```
node mark-applied.mjs
```

Then open **http://127.0.0.1:8900**. It runs only on your own Mac, so nobody else can reach it. Leave that Terminal window open while you use the dashboard.

TAB	WHAT IT SHOWS
New Jobs Found	Fresh roles from the latest scan, ranked by fit to your CV, with the reason for each score.
Applied Jobs	Everything in your tracker with its status and date.
All Jobs History	Every role ever discovered, deduplicated.
Archived	Roles you hid. Unarchive anytime.

Two buttons on each role

- **Mark Applied:** you applied yourself (e.g. on the company site). It adds the role to your tracker, once only.
- **Archive:** not interested. It hides the role from New Jobs.

"Source" vs "Apply" links.

"Source" is where a role was found (often LinkedIn). "Apply" is the employer's own page. LinkedIn listings are treated as leads only. Always apply on the employer's site when there is one.

Gmail sync

If you already sort job emails into Gmail labels, Claude can turn them into tracker updates: rejected, interview, offer.

a Create four labels in Gmail

Jobs/Applied , Jobs/Interview , Jobs/Rejected , Jobs/Offered . Then add Gmail filters (e.g. emails containing "unfortunately" or "not moving forward" → Jobs/Rejected), or label emails by hand as they arrive.

b Connect Gmail to Claude

Turn on Claude's Gmail connector for your account. Claude only *reads* the labelled emails. It never sends, deletes or relabels anything.

c Map your labels once

"List my Gmail labels and save the four Jobs/ label ids to data/gmail-labels.json."*

d Sync whenever you like

"Sync gmail."

Claude reads the new labelled emails, matches them to your tracker, and shows each proposed change with the email that supports it. Only the ones you approve are applied.

Labels are hints, not verdicts.

Newsletters and job alerts often end up in Jobs/Interview by accident, so an interview is only ever proposed from the email's actual text.

STEP 12

Applying to a job

You can always just apply yourself and click **Mark Applied**. For long forms, the assistant can fill them in for you to review. Simplest is to ask Claude:

"Fill the application for this job, but don't submit: <job link>"

Behind the scenes it runs these steps (you can also run them yourself):

```
# 1. open a visible browser window on the job
node browser-session.mjs start "<job link>"

# 2. check your answer bank and final CV
node application-runner.mjs --check-bank

# 3. show what it would fill (changes nothing)
node application-runner.mjs --plan --url "<job link>" --company "Acme"

# 4. fill the form, never submit
node application-runner.mjs --fill --url "<job link>" --company "Acme"
```

Then you

1. Read what it filled, and the list of questions it handed back to you.
2. Answer those yourself in the browser window.
3. Click Submit yourself.
4. Tell Claude *"I submitted it"* (or use Mark Applied) so the tracker records it.

Built-in brakes.

It stops at CAPTCHAs, logins and one-time codes, and legal declarations. It never overwrites something you already typed into a form, never fills in sensitive questions you haven't answered in your bank, and never uploads any CV except your verified final one.

Several roles at once? List them in `data/manual-targets.json` as `[{"company": "...", "role": "...", "url": "..."}]` and run `node open-for-manual.mjs`. Each one opens pre-filled in its own tab.

STEP 13

Your house rules

The file `modes/_custom.md` holds rules Claude follows in every session. Ask Claude to add these (and any of your own):

- Never submit an application without my explicit approval for that specific job.
- Never mark anything Applied because a form was opened or filled; only a site-confirmed submission or my own statement counts.
- Upload only my final CV (node final-cv.mjs); never generate or tailor one.
- Never invent answers. Anything not in data/application-answers.yml or cv.md is a question for me.
- Stop at CAPTCHAs, logins/OTPs and legal attestations.

Rules can also be personal preferences, e.g. *"Never suggest roles at companies under 20 people"* or *"Always show salary first in evaluations."* Changed your mind? Just tell Claude, and it updates the file.

Don't want to apply somewhere?

Add the company to `data/blacklist.md` (ask Claude). The scanner skips it from then on.

STEP 14

A normal day

WHEN	WHAT HAPPENS	YOU DO
11:30 & 20:00	The scan runs and a notification says how many new roles passed your filters.	Nothing.
Once a day	Open the dashboard. Archive what doesn't interest you.	~5 minutes.
Good matches	Ask Claude to evaluate one in depth: " <i>Evaluate this job: <link></i> ". You get a scored report with risks and talking points.	Decide: apply or skip.
Applying	Apply yourself, or let it fill the form and stop for you.	Review and submit.
Every few days	"Sync gmail" brings rejections and interviews into the tracker.	Approve the updates.

Useful things to ask Claude

- "*Show my application stats*": how many applied, response rate, interviews.
- "*Which applications need a follow-up?*"
- "*Prepare me for my interview at <company>*"
- "*Draft a LinkedIn message to the hiring manager for <role>*"
- "*Why am I getting rejected? Look for patterns.*"
- "*What skills keep coming up in roles I'm missing?*"

Quality over quantity.

The system is designed to help you send fewer, better applications. It will tell you plainly when a role scores below 4.0/5 and recommend against applying.

STEP 15

Staying up to date

To get improvements to this setup, run this in the `career-ops` folder (or ask Claude to "pull the latest version"):

```
git pull
```

Your CV, profile, answer bank, tracker and anything in `data/` and `output/` are never touched by an update. They're private to your machine.

STEP 16

Privacy: what stays where

STAYS ONLY ON YOUR MAC

`cv.md` , your CV PDF,
`config/profile.yml` , `portals.yml` ,
`modes/_profile.md` ,
`modes/_custom.md` , everything in
`data/` (tracker, answer bank, Gmail label
map, browser profile), `output/` , `.env`
keys

SHARED IN THE PROJECT (NO PERSONAL DATA)

The scripts, templates, tests and docs. None of them contain anyone's personal details. They read yours from the private files on the left.

- Job pages and forms are opened in a separate browser profile, never your everyday Chrome.
- Your data goes only to the job sites you apply on, and to Claude while you chat with it.
- If you share your own copy of the project later, ask Claude to scan it for personal data before you publish anything.

STEP 17

Troubleshooting

PROBLEM	FIX
command not found: node or brew	Close and reopen Terminal. If it persists, rerun the "Next steps" lines Homebrew printed at the end of its install.
npm install fails	Check <code>node -v</code> is 18.17 or newer: <code>brew upgrade node</code> .
Browser won't open / "executable doesn't exist"	<code>npx playwright install chromium</code>
No notifications	System Settings → Notifications → allow notifications for Script Editor / Terminal.
Scan finds 0 roles every time	Your title or location filters are too strict. Ask Claude: <i>"Why did the last scan find nothing? Show me what the filters rejected."</i>
Scores seem wrong	Make sure <code>modes/_profile.md</code> is personalised (step 4). Tell Claude <i>"this score is too high because..."</i> ; it learns from that.
Dashboard buttons do nothing	The server isn't running. Start <code>node mark-applied.mjs</code> and use <code>http://127.0.0.1:8900</code> , not the file directly.
"Address already in use" on 8900	The dashboard is already running. Open the link. Or use <code>node mark-applied.mjs --port 8901</code> .
<code>final-cv.mjs</code> says the CV is altered	The PDF changed since you recorded its hash. Rerun <code>shasum -a 256</code> and update <code>cv.final_sha256</code> .
The form filler asks too many questions	Fill more of your answer bank (step 6). Each answer you add is reused on every future form.
Using Linux instead of macOS	Schedule <code>bash daily-scan.sh</code> with cron. Notifications fall back to the log in <code>data/scan-cron.log</code> .
Anything else	Ask Claude in the project folder: <i>"Run the setup doctor and pipeline health check, and tell me what's wrong."</i>

You're set.

From here on, most things are one sentence to Claude away. When in doubt, describe what you want in plain words and it will find the right tool, and ask before doing anything it can't undo.